SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51
COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I Atchaya Chandran R(192521394) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

R. Atchaya

Question 1: RSA Key Generation and Security Analysis

Given Parameters

- Prime
- Prime
- Public exponent
- Plaintext message

Step 1: Compute Modulus and Euler's Totient

Step 2: Calculate the Private Key

The private key must satisfy the modular multiplicative inverse condition:

Using the **Extended Euclidean Algorithm**:

1.
2.
3.

Working backwards to express as a linear combination of and :

Taking modulo :

- **Verification:** .
- **Keys Summary:**
  - Public Key:
  - Private Key:

Step 3: Encrypt the Message ()

Ciphertext formula:

Step 4: Decrypt the Ciphertext () and Verify Correctness

Decrypted message formula:

Using the Chinese Remainder Theorem (CRT) for efficient modular exponentiation ():

1. **Modulo :**

1. **Modulo :**

Since and , by CRT:

- **Verification:** The decrypted message equals the original message .

Step 5: Security Analysis of Small Prime Numbers

- **Susceptibility to Factorization:** Selecting small primes () yields a tiny modulus (). An attacker can factor in microseconds using basic trial division or Pollard's rho algorithm to extract and , compute , and derive the private key .
- **Exhaustive Key Search:** Small key spaces allow trivial brute-force attacks.
- **Modern Best Practice:** Practical implementations require large, random prime numbers (at least 1024 to 2048 bits each, resulting in bits) to withstand modern factorization algorithms such as the **General Number Field Sieve (GNFS)**.

Question 2: ElGamal Encryption Analysis

Given Parameters

- Prime
- Generator
- Receiver's private key
- Ephemeral/Random key
- Plaintext message

Step 1: Compute the Public Key

- **Public Key:**

Step 2: Perform Encryption to Generate Ciphertext Pair

1. **Compute :**

1. **Compute Shared Secret Key :**

1. **Compute :**

- **Ciphertext Pair:**

Step 3: Decrypt the Ciphertext to Retrieve Message

1. **Recover Shared Key using Private Key :**

1. **Find Modular Inverse :**

Since , .

1. **Compute Decrypted Message :**

- **Result:** Message is successfully retrieved.

Step 4: Analysis of Randomness () in ElGamal Security

- **Probabilistic Encryption:** ElGamal is non-deterministic. Choosing a fresh random for each transaction produces completely different ciphertexts even if the same message is encrypted multiple times.
- **Resilience to Pattern Analysis:** Prevents attackers from conducting frequency analysis or detecting identical transmitted messages.
- **Semantic Security (IND-CPA):** Ephemeral randomness ensures an attacker cannot gain information about the plaintext from the ciphertext.
- **Caution:** Reusing the value across two different messages allows an adversary to deduce the ratio of plaintexts and compromise confidential communications.

Question 3: RSA Digital Signature Verification

Given Parameters

- Prime
- Prime
- Public exponent
- Message

Step 1: Compute Modulus and

Step 2: Find Private Key

Using standard modular inversion:

- **Verification:** .

Step 3: Generate the Digital Signature

Digital signatures are created using the sender's **private key** :

Breaking down into binary components:

Combining terms:

- **Digital Signature generated:**

Step 4: Verify the Signature Using Public Key

To verify, compute :

Using successive squaring:

- **Verification:** Recovered message , matching original . Signature is **valid**.

Step 5: Non-Repudiation Analysis

- **Exclusive Ownership:** Since only the legitimate owner holds the secret signing key , no external entity can forge a valid signature .
- **Public Verifiability:** Anyone can verify the signature using the corresponding public key .
- **Legal and Technical Binding:** Once a valid signature is verified against the public key, the sender cannot deny ("repudiate") originating the message.
- **Integrity Guarantee:** Any alteration to in transit causes signature verification to fail, protecting both authenticity and integrity.

Question 4: Diffie-Hellman Key Exchange with Attack Resistance Evaluation

Given Parameters

- Public prime
- Public generator
- User A private key
- User B private key

Step 1: Compute Public Keys for Both Users

- **User A Public Key ():**

- **User B Public Key ():**

Step 2: Compute the Shared Secret Key

- **Calculated by User A:**

- **Calculated by User B:**

- **Shared Secret Key:**

Step 3: Man-in-the-Middle (MITM) Attack Analysis

- **Vulnerability:** Standard Diffie-Hellman lacks identity verification.
- **Interception Scenario:**
  1. An active attacker (Mallory/Darth) intercepts User A's public key and replaces it with their own generated public key .
  2. Mallory intercepts User B's public key and replaces it with .
  3. User A establishes a secret key with Mallory (), and User B establishes a separate secret key with Mallory ().
- **Impact:** Mallory can transparently decrypt, read, modify, and re-encrypt all messages passing between User A and User B without detection.

Step 4: Cryptographic Solutions for Authenticity

1. **Digital Signatures / Public Key Infrastructure (PKI):** Each user signs their Diffie-Hellman public parameters using an authenticated private key verified via trusted Certificate Authorities (CAs).
2. **Station-to-Station (STS) Protocol:** Combines Diffie-Hellman key exchange with digital signatures and identity certificates.
3. **Pre-Shared Keys (PSK) / HMAC:** Utilizing pre-established credentials during authentication phases (e.g., TLS 1.3 or IPsec IKE).

Question 5: Hybrid Encryption System Analysis

Given Parameters

- RSA (Asymmetric) Key Exchange Time =
- AES (Symmetric) Data Block Encryption Time = per block
- Total Data Blocks =

Step 1: Calculate Total Time Taken for Hybrid Encryption

In a hybrid encryption system:

1. RSA encrypts the single symmetric AES key.
2. AES encrypts all 500 data blocks.

Step 2: Compare with Using Only RSA for Encrypting All Data

If RSA were used directly to encrypt all 500 data blocks individually:

Performance Comparison Table

Encryption Strategy	Key Distribution Time	Data Encryption Time	Total Time	Speedup Factor
RSA Only				(Baseline)
Hybrid System				**Faster**

- **Time Saved:** reduction in processing time (**performance gain**).

Step 3: Efficiency Analysis

- **Mathematical Operations:** RSA relies on modular exponentiations on large numbers, which are computationally expensive. AES uses hardware-accelerated substitution-permutation networks (SPN) operating on standard block sizes.
- **Algorithmic Complexity:** Asymmetric ciphers run orders of magnitude slower than symmetric ciphers for large volumes of data.
- **Hybrid Optimization:** By limiting asymmetric operations strictly to session key exchange, high execution overhead is incurred only once per session.

Step 4: Justification in Modern Secure Communication Systems

- **Scalability:** Enables high-bandwidth real-world applications (such as HTTPS, TLS 1.3, SSH, PGP, and IPsec) to encrypt gigabytes of data with negligible latency.
- **Key Distribution:** Solves the symmetric key distribution challenge without requiring a secure pre-shared secret channel.
- **Security & Confidentiality:** Delivers fast symmetric payload processing alongside asymmetric key exchange and Perfect Forward Secrecy (PFS).